Exercise 06: *PyUVM* Environment, Sequencer, and Sequences

Objective: *Introduction to PyUVM environments, sequencers and sequences.*

Task: *Expanding the existing PyUVM tests for the Saturation Filter filter with a PyUVM environment virtual sequencer and virtual sequences.*

In Part 3 of exercise 05 we ended with the *PyUVM* test class, directly driving the inputs of the DUT though the *sSDT* interfaces in the `run_phase` (See Figure 1). This this exercise extends the testbench setup with a *PyUVM* environment, virtual sequencer, and virtual sequence.

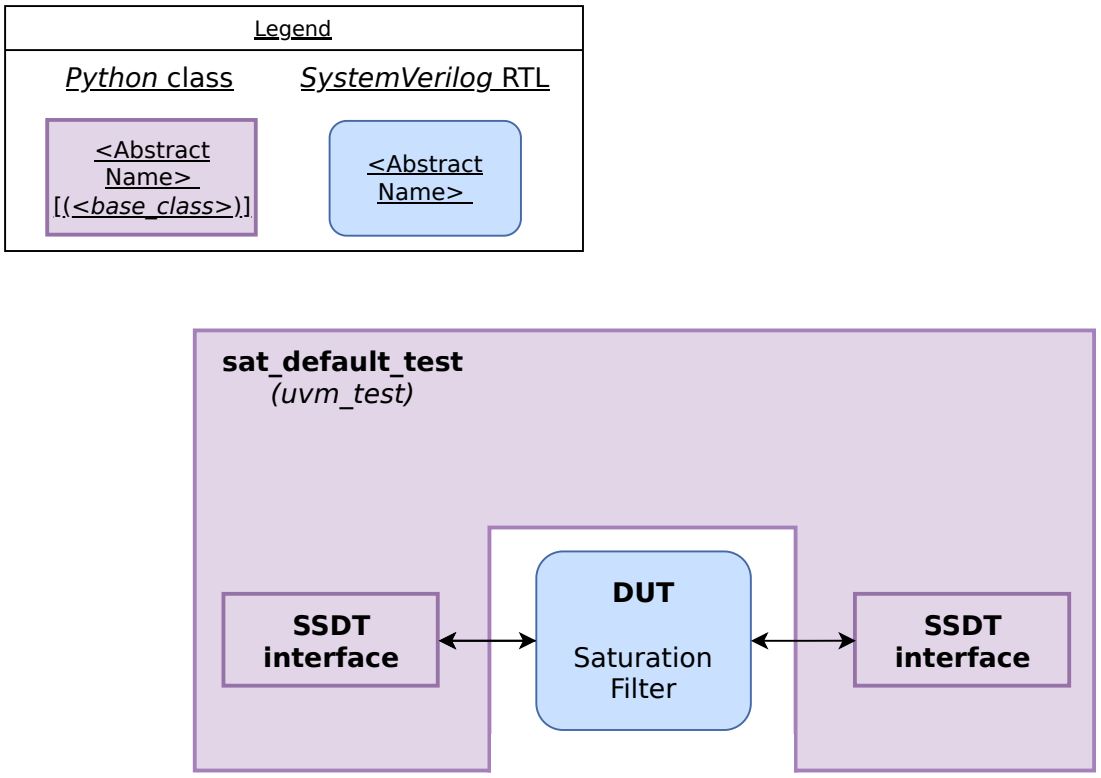


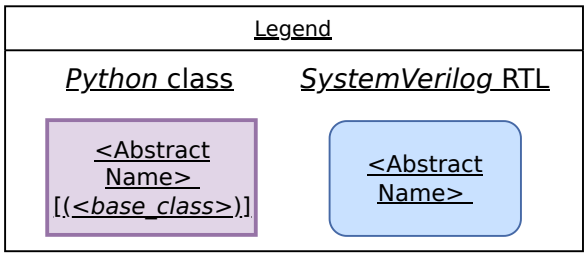
Fig. 1: Testbench setup from *Exercise 05 Part3: Interface Wrapper Test.* The Interfaces are driven driven directly from the test class.

We will extend the testbench in three steps. See Figure 2 for the testbench setup, that will be in place after this exercise. After setting up the new classes and default test, we will recreate the random test from *Exercise 05 Part 2*.

- In Part 1: Virtual Sequencer and Environment we extend the test with the virtual sequencer and environment.
- In Part 2: Testbench Configuration we setup the testbench configuration object and send it to the virtual sequencer via the `ConfigDB`.
- In Part 3: Virtual Top Sequence we create a virtual sequence to drive the inputs of the DUT via the interfaces.
- In Part 4: Random Test we create a second testcase and virtual sequence to drive random stimuli.

Note:
This exercise introduces a number of new classes and storing all of them in a single source file would make the code illegible, therefore from now on we will try to limit the number of class definitions per source file to one.

`lecture04/exercises/E06_sat_pyuvm_env_vseq/tb` contains the `Makefile` and the base files for all classes implemented in this exercise. In the following sections the base directory of the files is omitted.



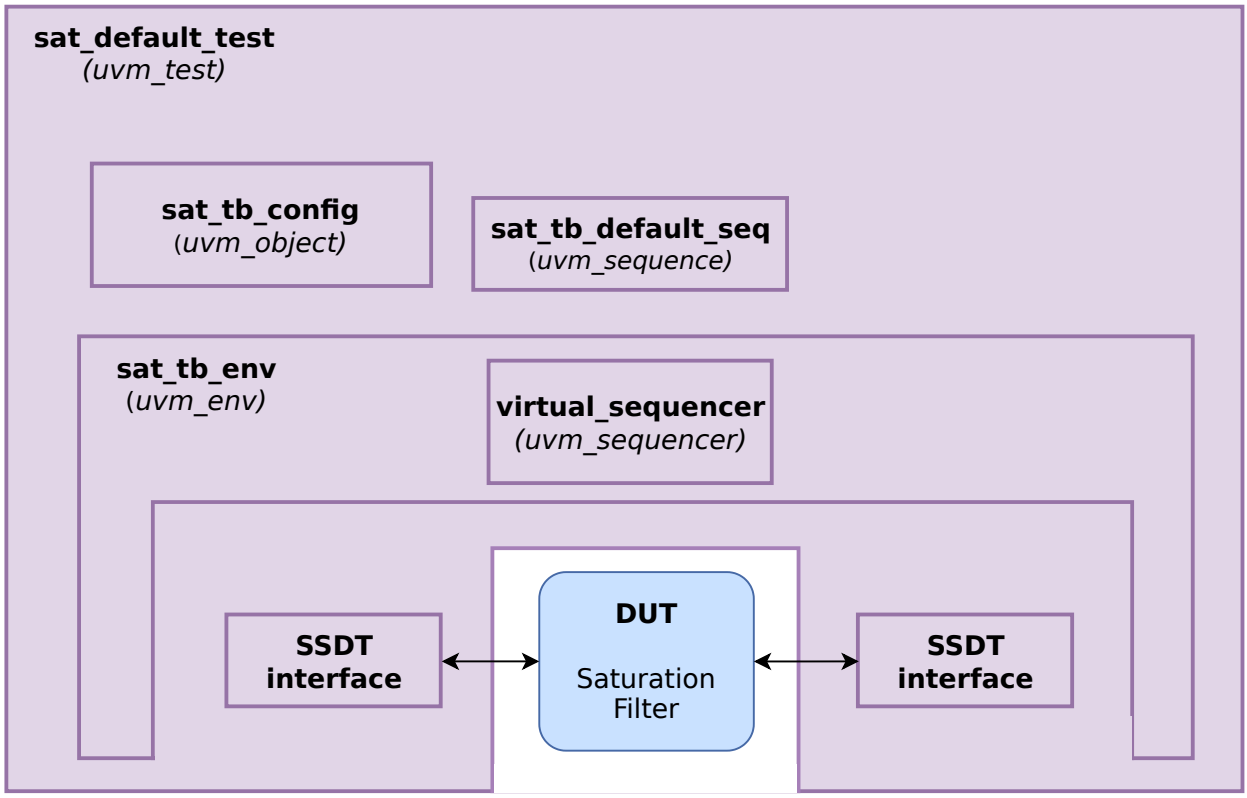


Fig. 2: Testbench overview. This exercise implements this class structure for the *Saturation Filter PyUVM* testbench.

Part 1: Virtual Sequencer and Environment

First we need to create testbench environment and the virtual sequencer.

In `sat_tb_virt_sequencer.py` create a virtual sequencer class, extending `uvm_sequencer`.

- Overwrite the class constructor (`__init__`):
  - Set a default name for the class.
  - Create an empty handle `cfg` for the configuration object.
  - Don't forget to call `super().__init__(<args>)`.

In `sat_tb_env.py` create a new environment class extending `uvm_env`.

- Overwrite the class constructor:
  - Set a default name for the class.
  - Create an empty handle `cfg` for the configuration object.
  - Create an empty handle `virtual_sequencer` for the virtual sequencer.
- In the `build_phase`:
  - Instantiate a virtual sequencer using the factory with `<class>.create()`. See below for more information on the factory class creation.

The UVM Factory

All classes extending `uvm_object` or `uvm_component` are automatically registered in the `uvm_factory`. To create an object using the factory the `create()` class method is used. A `uvm_component` is created by calling:

```
component = <component_class>.create(
  name = "<component_name>",
  parent = <parent component in hierarchy>
)
```

A `uvm_object` does not have a parent component and therefore is created by calling:

```
object = <object_class>.create(
  name = "<object_name>",
)
```

Part 2: Testbench Configuration

Next we need to create is the testbench configuration. The config typically contains parameters to adapt the testbench to changing DUT parameters. It is also used to share the DUT interface classes. This is necessary, since the stimuli generation will be moved into a sequence and the sequences does not have direct access to the interfaces.

Creation and Instantiation of the Configuration Object

In `sat_tb_config.py` create a testbench configuration object, extending `uvm_object`.

- Overwrite the class constructor:
  - Set a default name for the class
  - Create an empty handle `input_if` for the input interface.
  - Create an empty handle `output_if` for the output interface.

In `sat_default_test.py` create a new *PyUVM* test.

- In the `build_phase`:
  - Instantiate a `ssdt_interface_wrapper` for the input interface.
  - Instantiate a `ssdt_interface_wrapper` for the output interface.
  - Create an instance of the configuration class from `sat_tb_config.py` using the factory. Hint: The configuration is a `uvm_object` and therefore has no parent.
  - Assign the interfaces to the interface handles in the configuration.

```
self.cfg.input_if = self.input_if
```

- In the `connect_phase`:
  - The input interface to the `clk`, `rst`, `in_valid`, and `in_data` signals of the DUT.
  - The output interface to the `clk`, `rst`, `out_valid`, and `out_data` signals of the DUT.

Sharing the Configuration with the Environment and Virtual Sequencer

We will need access the configuration object in the environment and the virtual sequencer. To share objects between parts of the testbench we use the `ConfigDB`. The `ConfigDB` is a scoped dictionary used to pass variables between components without direct connections. A value is stored with with `ConfigDB().set(context, inst_name, field_name, value)` and retrieved with `ConfigDB().get(context, inst_name, field_name)`. It is important to note that `inst_name` is always relative to the full instance path of the component provided as context. See the method descriptions below for more information.

CongfigDB set()

```
def set(context, inst_name, field_name, value)
```

Stores an object in the db using the context and `inst_name` to create a retrieval path, and the key name. The `inst_name` is relative to the context instance path of the context component.

Arguments:

- `context`: A handle to a component
- `inst_name`: The instance name within the component
- `field_name`: The key we're setting
- `value`: The object to be stored
- Returns: None

CongfigDB get()

```
def get(context, inst_name, field_name, default=default_get)
```

The component path matches against the paths in the `ConfigDB`. The path cannot have wildcards, but can match against keys with wildcards. Return the value stored at key. The `inst_name` is relative to the context instance path of the context component.

Arguments:

- `context`: The component making the call
- `inst_name`: component full path with no wildcards
- `field_name`: the `field_name` being retrieved
- `default`: the value to return if there is no key
- *returns:* value found at location

In `sat_default_test.py`

- In the `build_phase`:
  - Create an instance of the environment using the factory.
  - Set the `cfg` field of `ConfigDB` for the instance of the environment object to the handle of the configuration object.

```
ConfigDB().set(
    context     = self,
    inst_name   = 'sat_tb_env',
    field_name  = 'cfg',
    value       = self.cfg,
)
```

In `sat_tb_env.py`:

- In the `build_phase`:
  - Get the configuration object from the `ConfigDB`. Note that `inst_name` is relative to the context.

```
self.cfg = ConfigDB().get(
    context    = self,
    inst_name  = "",
    field_name = "cfg",
)
```

  - Set the `cfg` field of `ConfigDB` for the instance of the virtual sequencer object to the handle of the configuration object.

In `sat_tb_virt_sequencer.py`

- In the `build_phase`:
  - Get the configuration object from the `ConfigDB`.

Part 3: Virtual Top Sequence

With the new testbench infrastructure in place, we can now setup the stimuli. You can refactor most your code from Part 3 of Exercise 05.

In `sat_tb_default_test.py`

- In the `run_phase`:
  - Start a clock on `clk`.
  - Reset the DUT using `rst`.

We are now driving the stimuli from a virtual sequence. The virtual sequence is going to run on the virtual sequencer we created previously. Every `uvm_sequence` by default has a handle to the sequencer running the sequence.[1]

Creating the Virtual Sequence

First we are going to create the virtual sequence. In `sat_tb_default_seq.py` create a sequence class extending `uvm_sequence`.

- Overwrite the class constructor:
  - Set a default name for the class.
  - Create an empty handle `sequencer`, for the handle to the sequencer.
  - Create an empty handle `cfg`, for the handle to the configuration object.
- In the `pre_body`:
  - Assign the configuration object from the sequencer to the `cfg` handle.

```
self.cfg = self.sequencer.cfg
```

  - Optionally the interface handles can also be assigned to local instance variables for easier use.

```
self.input_if = self.cfg.input_if
self.output_if = self.cfg.output_if
```

- In the `body`:
  - Refactor your stimuli generation from Exercise 05.

[1] You can inspect the source code `uvm_sequence.start()` to see how the sequence handle is propagated

Running the Virual Sequence on the Virtual Sequencer

Lastly we need to start the virtual sequence on the virtual sequencer. In `sat_default_test.py`

- In the `run_phase`:
  - Create an instance of the virtual sequence using the factory.
  - Start the virtual sequence on the virtual sequencer. You can access the virtual sequencer though the environment handle.

```
sequence>.start(seqr=self.<env_handle>.<sequencer_handle>)
```

Part 4: Random Test

Refactor the Random Test from Exercise 05 into a virtual sequence In `sat_tb_random_seq.py` create a `uvm_sequence` extending the default sequence.

- Overwrite the class constructor:
  - Set a default name for the class.
- Overwrite the `body`:
  - Refactor your `sat_random_test` from Exercise 05.

In `sat_random_test.py` create a new test case extending the default test.

- In `start_of_simulation_phase`:
  - Create a *factory type override by type* to replace the default sequence with the random sequence. See below for more information of factory overrides.

```
uvm_factory().set_type_override_by_type(
    original_type=sat_tb_default_seq,
    override_type=sat_tb_random_seq,
)
```

Factory Overrides

Overrides allow us to replace any object registered in the factory without modifying the existing code. Overrides can either be:

- Per type, meaning the entire type (class) is replaced with the new type.
- Per instance, meaning only types with the provided instance path are replaced with the new type. The instance path can contain wildcards e.g. `"sat_default_test.tb_env*"` matches any path starting with `"sat_default_test.tb_env"`.

Factory type override by type

```
def set_type_override_by_type(original_type, override_type, replace=True)
```

Override one type with another type globally

- `original_type`: The original type to be overridden
- `override_type`: The new type that will override it
- `replace`: If the override exists, only replace it if this is True
- returns: None

Factory instance override by type

```
def set_inst_override_by_type(original_type, override_type, full_inst_path)
```

Override an instance with a new type if original type is at that path

- `original_type`: The original type being overridden
- `override_type`: The overriding type
- `full_inst_path`: The inst where this happens
- returns: None